Міністерство освіти і науки України
Тернопільський технічний національний університет ім. І.Пулюя

Кафедра комп’ютерних
систем та мереж

Звіт

Про виконання лабораторної роботи №4
З дисципліни «Програмування мовою Java»
На тему: «Розробка структури класів
з урахуванням зазначеної предметної області»

Виконав:
Студент групи СІ-21
Статок Арсеній Сергійович
Перевірив:
Луцків Андрій Мирославович

Тернопіль 2026

Мета. Навчитись розробляти структури класів з урахуванням зазначеної предметної області

Завдання:

В програмі показати використання внутрішніх(inner), вкладених(staticnested) та локальних класів. Показати зв'язки між класами: асоціація, композиція, агрегація. Реалізувати мінімальну взаємодію між екземплярами класів.

23. Магазин електроніки

ХІД РОБОТИ

Проєкт моделює роботу магазину електроніки та складається з двох пакетів: enums — з перерахуваннями статусів клієнта, замовлення, посад та типів тікетів, і model — з основними класами-сутностями (Customer, Order, OrderItem, Product, Staff, SupportTicket). Класи пов'язані між собою через композицію, агрегацію та асоціацію, а також містять вкладені класи трьох видів: нестатичний inner клас ActivityTracker всередині Customer, статичний nested клас Builder всередині Order, та локальний клас ReportFormatter всередині методу.

У класі Main послідовно демонструється робота всієї системи: спочатку створюється клієнт і через inner клас ActivityTracker симулюється 5 входів у систему, після чого статус клієнта автоматично підвищується до «Постійний». Далі створюються товари та через Order.Builder (static nested клас) будується замовлення з автоматичним розрахунком знижки й ПДВ, до якого додаються позиції OrderItem. Потім замовлення проходить через усі статуси — від «Нове» до «Завершено». Після цього демонструється асоціація: менеджер Staff призначається на тікет SupportTicket, який вирішується з оцінкою. Наприкінці локальний клас ReportFormatter форматує та виводить підсумкову квитанцію замовлення (рис. 1).

```java
Main.java
import enums.*;
import model.*;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

public class Main {

    public static void main(String[] argv) {

        System.out.println("┌──────────────────────────────────────┐");
        System.out.println("│    МАГАЗИН ЕЛЕКТРОНІКИ — ДЕМОНСТРАЦІЯ    │");
        System.out.println("└──────────────────────────────────────┘\n");

        // ── 1. Створення клієнта ──────────────────────────────────────
        Customer customer = new Customer(
                firstName: "Олена",  lastName: "Ковальчук",  middleName: "Іванівна",
                phone: "+380501234567",  email: "o.kovalchuk@mail.com",
                LocalDate.of(year: 1995,  month: 3,  dayOfMonth: 15),  gender: 'Ж'
        );
        customer.setCumulativeDiscount(5.0);
        System.out.println("Клієнт: " + customer);

        // ── 2. INNER CLASS — ActivityTracker ──────────────────────────
        // Inner class існує в контексті конкретного екземпляра Customer
        System.out.println("\n-- Inner class: ActivityTracker ---");
        Customer.ActivityTracker tracker = customer.createActivityTracker();
        tracker.recordLogin();
        tracker.recordLogin();
        tracker.recordLogin();
        tracker.recordLogin();
        tracker.recordLogin(); // 5-й вхід — статус підвищується до LOYAL
        tracker.printActivity();
        System.out.println("Статус після активності: " + customer.getStatus().getDisplayName());

        // ── 3. Товари ─────────────────────────────────────────────────
        System.out.println("\n-- Товари ---");
        Product laptop = new Product(
                name: "Ноутбук Dell XPS 15",  sku: "DELL-XPS15-001",
                basePrice: 45000.00,  category: "Ноутбуки",  manufacturer: "Dell",  warrantyMonths: 24
        );
        Product mouse = new Product(
                name: "Миша Logitech MX Master 3",  sku: "LOG-MX3-002",
                basePrice: 2500.00,  category: "Периферія",  manufacturer: "Logitech",  warrantyMonths: 12
        );
        System.out.println(laptop);
        System.out.println(mouse);

        // ── 4. STATIC NESTED CLASS — Order.Builder ────────────────────
        // Builder є static nested — не потребує екземпляра Order для створення
        System.out.println("\n--  Static nested class: Order.Builder ---");
        Order order = new Order.Builder(customer)   // аргумент: customer живе окремо
                .discount(0.05)
                .tax(0.20)
                .paymentMethod(PaymentMethod.CARD)
                .comment("Терміново замовлення, прошу відправити сьогодні")
                .build();

        // КОМПОЗИЦІЯ — OrderItem'и створюються всередині Order і без нього не існують
        order.addItem(laptop,  quantity: 1);
        order.addItem(mouse,  quantity: 2);

        System.out.println(order);
        System.out.println("Позиції замовлення (Композиція):");
        order.getItems().forEach(item -> System.out.println("  " + item));

        // ── 5. Оновлення статусу замовлення ───────────────────────────
        System.out.println("\n-- Обробка замовлення ---");
        order.setStatus(OrderStatus.PROCESSING);
        System.out.println("Статус: " + order.getStatus().getDisplayName());

        order.setStatus(OrderStatus.SHIPPED);
        order.setTrackingNumber("1Z345A7890123456");
        order.setShippedAt(LocalDateTime.now());
        System.out.println("Статус: " + order.getStatus().getDisplayName()
                + " | ТТН: " + order.getTrackingNumber());

        order.setStatus(OrderStatus.COMPLETED);
        order.setReceivedAt(LocalDateTime.now());
        System.out.println("Статус: " + order.getStatus().getDisplayName());

        // ── 6. Персонал + АСОЦІАЦІЯ зі SupportTicket ──────────────────
        System.out.println("\n-- Асоціація: Staff m SupportTicket ---");
        Staff manager = new Staff(
                firstName: "Михайло",  lastName: "Бондаренко",
                phone: "+380671112233",  email: "m.bondarenko@store.com",
                Position.MANAGER
        );
        System.out.println("Посада: " + manager.getPosition().getTitle()
                + " | Обов'язки: " + manager.getPosition().getDuties());

        SupportTicket ticket = new SupportTicket(
                customer, TicketType.COMPLAINT,
                description: "Ноутбук має подряпину на кришці після доставки"
        );
        System.out.println("\nТікет до призначення: " + ticket);

        // АСОЦІАЦІЯ — manager призначається на ticket; обидва незалежні
        ticket.assignManager(manager);
        System.out.println("Тікет після призначення: " + ticket);

        ticket.resolve(rating: 4);
        System.out.println("Тікет вирішено. Оцінка: " + ticket.getRating() + "/5");

        // ── 7. LOCAL CLASS — форматування звіту ───────────────────────
        System.out.println("\n-- Local class: ReportFormatter ---");
        printOrderReport(order);
    }

    // Метод із LOCAL CLASS всередині
    private static void printOrderReport(Order order) {  1 usage

        // ==========================================================
        // LOCAL CLASS (локальний клас всередині методу)
        // Видимий лише в межах методу printOrderReport
        // ==========================================================
        class ReportFormatter {  2 usages
            private final String separator = "=".repeat(count: 60);  4 usages

            public void printHeader(String title) {  1 usage
                System.out.println(separator);
                System.out.println("  " + title);
                System.out.println(separator);
            }

            public void printRow(String label, String value) {  6 usages
                System.out.printf("  %-22s: %s%n", label, value);
            }

            public void printItems(List<OrderItem> items) {  1 usage
                System.out.println("  Позиції:");
                for (int i = 0; i < items.size(); i++) {
                    OrderItem item = items.get(i);
                    System.out.printf("   %d. %-28s x%d  = %.2f ШАН%n",
                            i + 1,
                            item.getProduct().getName(),
                            item.getQuantity(),
                            item.getSubtotal());
                }
            }

            public void printTotal(double amount) {  1 usage
                System.out.println(separator);
                System.out.printf("  %-22s: %.2f ШАН%n", "РАЗОМ (з ПДВ 20%)", amount);
                System.out.println(separator);
            }
        }
        // ==========================================================

        // Використання local class
        ReportFormatter fmt = new ReportFormatter();
        fmt.printHeader("ВИТЯГ ЗАМОВЛЕННЯ");
        fmt.printRow("Клієнт",
                value: order.getCustomer().getLastName() + " "
                        + order.getCustomer().getFirstName());
        fmt.printRow("Email", order.getCustomer().getEmail());
        fmt.printRow("Статус", order.getStatus().getDisplayName());
        fmt.printRow("Оплата", order.getPaymentMethod().getDisplayName());
        fmt.printRow("ТТН", order.getTrackingNumber() != null ? order.getTrackingNumber() : "—");
        fmt.printRow("Коментар", order.getComment());
        fmt.printItems(order.getItems());
        fmt.printTotal(order.getTotalAmount());
    }
}
```

Рисунок 1 — Код програми

У результаті компіляції та виконання коду було виведено в консоль демонстраційний варіант, у якому показано роботу трьох видів вкладених класів (рис. 2).

```
C:\Users\User\.jdks\openjdk-25.0.2\bin\java.exe "-javaagent:D:\IntellIJI\IntelliJ IDEA 2025.3.2\lib\idea_rt.jar=63698"
╔═══════════════════════════════════╗
║     МАГАЗИН ЕЛЕКТРОНІКИ — ДЕМОНСТРАЦІЯ    ║
╚═══════════════════════════════════╝


Клієнт: Customer{Олена Коваленко, email='o.kovalenko@email.com', status=Звичайний, discount=5,0%}

--- Inner class: ActivityTracker ---
=== Журнал активності: Олена Коваленко ===
[2026-03-18T12:59:41.659350] Логін #1 клієнта: Олена Коваленко
[2026-03-18T12:59:41.667351900] Логін #2 клієнта: Олена Коваленко
[2026-03-18T12:59:41.667351900] Логін #3 клієнта: Олена Коваленко
[2026-03-18T12:59:41.667351900] Логін #4 клієнта: Олена Коваленко
[2026-03-18T12:59:41.667351900] Логін #5 клієнта: Олена Коваленко
Статус підвищено до LOYAL після 5 входів.
Всього входів: 5
Статус після активності: Постійний

--- Товари ---
Product{name='Ноутбук Dell XPS 15', sku='DELL-XPS15-001', price=45000,00, manufacturer='Dell'}
Product{name='Мишка Logitech MX Master 3', sku='LOG-MX3-002', price=2500,00, manufacturer='Logitech'}

--- Static nested class: Order.Builder ---
Order{customer='Олена Коваленко', status=Нове, total=57000,00 UAH, items=2}
Позиції замовлення (Композиція):
  OrderItem{product='Ноутбук Dell XPS 15', qty=1, price=42750,00, subtotal=42750,00}
  OrderItem{product='Мишка Logitech MX Master 3', qty=2, price=2375,00, subtotal=4750,00}

--- Обробка замовлення ---
Статус: В обробці
Статус: Відправлено | ТТН: 1234567890123456
Статус: Завершено

--- Асоціація: Staff ↔ SupportTicket ---
Staff{Михайло Бондаренко, position='Менеджер'}
Посада: Менеджер | Обов'язки: Обробка замовлень та підтримка клієнтів

Тікет до призначення: SupportTicket{type=Скарга, status='OPEN', customer='Олена Коваленко', manager='не призначено'}
Тікет після призначення: SupportTicket{type=Скарга, status='IN_PROGRESS', customer='Олена Коваленко', manager='Михайло Бондаренко'}
Тікет вирішено. Оцінка: 4/5

--- Local class: ReportFormatter ---
────────────────────────────────────────────
  КВИТАНЦІЯ ЗАМОВЛЕННЯ
────────────────────────────────────────────
  Клієнт                : Коваленко Олена
  Email                 : o.kovalenko@email.com
  Статус                : Завершено
  Оплата                : Картка
  ТТН                   : 1234567890123456
  Коментар              : Термінове замовлення, прошу відправити сьогодні
  Позиції:
    1. Ноутбук Dell XPS 15          x1  = 42750,00 UAH
    2. Мишка Logitech MX Master 3   x2  = 4750,00 UAH
────────────────────────────────────────────
  РАЗОМ (з ПДВ 20%)     : 57000,00 UAH
────────────────────────────────────────────


Process finished with exit code 0
```

Рисунок 2 — Результат виконання програми

ВИСНОВКИ

У ході виконання лабораторної роботи було реалізовано об'єктно-орієнтовану модель магазину електроніки на Java 17, в якій практично продемонстровано три види вкладених класів: inner клас із прямим доступом до полів зовнішнього екземпляра, static nested клас для реалізації патерну Builder без прив'язки до екземпляра, та local клас для інкапсуляції логіки форматування всередині методу; окрім цього, на прикладі класів Order, Customer та SupportTicket наочно показано відмінності між композицією, агрегацією та асоціацією як основними видами зв'язків між класами в об'єктно-орієнтованому програмуванні.